Bitácora de TareasYA

Registro de cambios, mejoras y sucesos - julio a noviembre de 2026

Autor: Santino Crespo - 6.º 2.ª - E.T. N.º 35 Ing. Eduardo Latzina

Versión actual	Período	Versiones	Tests
v0.11.0 (3 de noviembre de 2026)	5 de julio al 3 de noviembre	16 (11 reales, 5 simuladas)	53 (31 backend + 22 frontend), todos correctos

Cómo leer esta bitácora. Los hitos con fecha precisa y marcados como reales salen de los archivos de las entregas (TP 1, 2, 4, 5, 6 y 7) y del trabajo hecho sobre el repositorio. Las versiones marcadas como simuladas son una reconstrucción verosímil de los pasos intermedios, no un historial real de commits.

Etiquetas: **[Nuevo]** **[Mejora]** **[Fix]** **[Suceso]** **[Deuda]**

Resumen por versión

Versión	Fecha	Tipo	Título
v0.1.0	5 de julio de 2026	real	Diseño (TP 1)
v0.2.0	mediados de julio	simulada	Prototipo
v0.2.1	fines de julio	simulada	Correcciones del prototipo
v0.3.0	principios de agosto	simulada	Primera ronda del challenge
v0.4.0	19 de agosto de 2026	real	Refactoring + tests unitarios (TP 2)
v0.5.0	septiembre de 2026	simulada	Migración a backend
v0.5.1	mediados de septiembre	simulada	Persistencia en archivo
v0.6.0	30 de septiembre de 2026	real	Backend (TP 4)
v0.6.1	30 de septiembre de 2026	real	Analizar antes de pedir (TP 5)
v0.7.0	19 de octubre de 2026	real	Mejora del agente revisor (TP 6)
v0.7.1	19 de octubre de 2026	real	Medí tu revisor (TP 7)
v0.8.0	3 de noviembre de 2026	real	Repositorio unificado
v0.9.0	3 de noviembre de 2026	real	Backend completo y endurecido
v0.10.0	3 de noviembre de 2026	real	Aplicación web completa
v0.10.1	3 de noviembre de 2026	real	Documentación
v0.11.0	3 de noviembre de 2026	real	Ciclo Rojo-Verde-Refactor, endpoints y agentes

Detalle versión por versión

v0.1.0 - 5 de julio de 2026

Diseño (TP 1)

- **[Suceso]** Arranca el proyecto con el TP 1: Diseñá tu propia aplicación.
- **[Nuevo]** Definición del problema: estudiantes que pierden entregas por manejar tareas en cuaderno o chat del curso.
- **[Nuevo]** SPEC inicial: objetivo, usuario, funcionalidades, restricciones y tecnología (React + localStorage).
- **[Nuevo]** Modelo de datos: título, descripción, materia, fecha límite, prioridad, estado, favorito, fecha de creación.
- **[Nuevo]** Prompt profesional y challenge de validación de 7 puntos para revisar el código generado.

v0.2.0 - mediados de julio (simulada)

Prototipo

- **[Nuevo]** Primer prototipo React: alta de tareas con persistencia en localStorage y lista básica.
- **[Deuda]** Validación y creación de la tarea viven dentro del componente del formulario.

v0.2.1 - fines de julio (simulada)

Correcciones del prototipo

- **[Fix]** El formulario no se vaciaba tras crear una tarea.
- **[Mejora]** Mensajes de error visibles cuando faltan datos obligatorios.

v0.3.0 - principios de agosto (simulada)

Primera ronda del challenge

- **[Nuevo]** Estados (pendiente, en curso, completada) y marca de favorito.
- **[Mejora]** Primera pasada contra el challenge de validación del TP 1.
- **[Deuda]** TaskForm.jsx mezcla UI, validación, armado del objeto y guardado.

v0.4.0 - 19 de agosto de 2026

Refactoring + tests unitarios (TP 2)

- **[Suceso]** Se refactoriza TaskForm.jsx; la versión previa queda en docs/TaskForm.before-refactor.jsx.
- **[Mejora]** Se extrae taskLogic.js: lógica pura, sin React ni DOM (validarDatosTarea, generarIdTarea, crearTarea).
- **[Mejora]** TaskForm queda solo con estado de UI y delega en la lógica.
- **[Nuevo]** taskLogic.test.js con Vitest: 8 tests (5 de validación, 3 de creación), con casos correctos, inválidos y límite.

v0.5.0 - septiembre de 2026 (simulada)

Migración a backend

- **[Suceso]** Se decide que qué es una tarea válida no puede vivir solo en el navegador.
- **[Nuevo]** Servidor Express en el puerto 3001 con arquitectura en capas: routes, controllers, services, repositories.
- **[Mejora]** La lógica del TP 2 se migra a backend/src/logic/taskLogic.js (versión CommonJS).

v0.5.1 - mediados de septiembre (simulada)

Persistencia en archivo

- **[Nuevo]** Repositorio con persistencia en data/tareas.json (equivalente backend de localStorage), creado solo si no existe.
- **[Nuevo]** Primera tarea de prueba cargada: TP de Física, prioridad alta.

v0.6.0 - 30 de septiembre de 2026

Backend (TP 4)

- **[Suceso]** Entrega del TP 4.
- **[Nuevo]** POST /api/tareas (201) y GET /api/tareas.
- **[Nuevo]** Regla de negocio propia: no se permiten dos tareas activas con el mismo título y materia (409); las completadas no cuentan.
- **[Mejora]** El controller solo traduce HTTP; las reglas quedan en el service.
- **[Deuda]** El backend unifica fecha vacía y fecha inválida en un solo mensaje.

v0.6.1 - 30 de septiembre de 2026

Analizar antes de pedir (TP 5)

- **[Suceso]** Ejercicio sobre queryTicketRows() (GET /tickets), código ajeno al proyecto para entrenar la mirada crítica.
- **[Mejora]** Criterio incorporado: antes de pedirle algo a la IA, leer qué decisiones quedaron implícitas (límites ocultos, errores silenciosos, volumen).
- **[Suceso]** Tres problemas ordenados por gravedad, probabilidad y detectabilidad: SQL como texto, JSON cortado a 8000 caracteres y lectura completa de la tabla en cada request.

v0.7.0 - 19 de octubre de 2026

Mejora del agente revisor (TP 6)

- **[Suceso]** Se afina el subagente revisor (solo lectura) armado en clase.
- **[Mejora]** Cinco cambios: description con más disparadores; alcance definido al git diff; quinta pregunta sobre secretos; formato fijo de hallazgo; salida para el caso limpio.

v0.7.1 - 19 de octubre de 2026

Medí tu revisor (TP 7)

- **[Suceso]** Se diseña una evaluación con 5 archivos: 3 con problema (login.py, subir.py, config.py) y 2 sanos (precios.py, clientes.py).
- **[Nuevo]** Expectativas escritas antes de correr el revisor, para que el puntaje mida algo real.

v0.8.0 - 3 de noviembre de 2026

Repositorio unificado

- **[Suceso]** Se reúnen los TP en un solo repositorio: backend/, frontend/, docs/.
- **[Nuevo]** taskStorage.js, README, .gitignore y scripts start y dev.

v0.9.0 - 3 de noviembre de 2026

Backend completo y endurecido

- **[Nuevo]** API REST completa: GET /tareas con filtros (q, materia, estado, prioridad, favorito) y orden (fecha, prioridad, creación); GET /tareas/:id; PATCH /tareas/:id; DELETE /tareas/:id; GET /salud.
- **[Nuevo]** Campo fechaActualizacion y validación parcial para ediciones (solo campos permitidos; rechaza id y otros no editables).
- **[Mejora]** app.js separado de server.js: la app se puede testear sin abrir un puerto (resuelve la deuda sobre listen al importar).
- **[Fix]** Mensajes distintos para fecha vacía (obligatoria) y fecha inválida (resuelve la deuda del TP 4).
- **[Fix]** Un título que no es texto ya no rompe la comparación de duplicados: se válida el tipo antes.
- **[Mejora]** Límites de largo (título 100, descripción 500) y tamaño máximo del cuerpo (100 kb).
- **[Mejora]** La regla 409 también se aplica al editar, ignorando la propia tarea.
- **[Mejora]** Repositorio con escritura atómica (archivo temporal + rename); una base dañada responde 500 genérico en vez de devolver una lista vacía y perder datos.
- **[Mejora]** Manejo central de errores: JSON invalido 400, cuerpo grande 413, rutas inexistentes 404; sin detalles internos.
- **[Nuevo]** 14 tests con el runner nativo de Node (lógica pura + API con base temporal). Resultado: 14 de 14 correctos.

v0.10.0 - 3 de noviembre de 2026

Aplicación web completa

- **[Nuevo]** App React con Vite: formulario de alta y edición, lista de tareas, resumen (pendientes, en curso, completadas, vencidas).
- **[Nuevo]** Búsqueda por título o materia, filtros por materia, estado, prioridad y favoritas, y orden por fecha, prioridad o recientes.
- **[Nuevo]** Acciones por tarea: cambiar estado, marcar favorita, editar y eliminar con confirmación.
- **[Nuevo]** Aviso de tareas que vencen en 48 horas y notificaciones del navegador (con permiso del usuario).
- **[Nuevo]** taskClient.js: usa la API si el servidor responde y pasa a modo local (localStorage) si no, con un aviso visible.
- **[Mejora]** taskLogic.js del frontend ampliado: aplicarCambios, existeDuplicada, filtrar, ordenar, vencimientos y resumen.
- **[Mejora]** Diseño responsive con modo oscuro automático, indicadores de prioridad por color, roles ARIA y foco visible.
- **[Mejora]** Proxy de Vite hacia el backend: sin CORS en desarrollo.
- **[Nuevo]** 22 tests de Vitest (antes 8). Resultado: 22 de 22 correctos y build de producción exitoso.

v0.10.1 - 3 de noviembre de 2026

Documentación

- **[Nuevo]** SPEC y plan técnico del sistema en PDF (docs/SPEC-TareasYA.pdf).
- **[Nuevo]** Esta bitácora en PDF y Markdown.
- **[Mejora]** README actualizado con la API completa, los comandos de prueba y el modo local.

v0.11.0 - 3 de noviembre de 2026

Ciclo Rojo-Verde-Refactor, endpoints y agentes

- **[Suceso]** Se adopta el ciclo Rojo-Verde-Refactor como forma de trabajo: dos ciclos completos, con un commit por fase y las salidas reales de los tests guardadas en docs/TDD-ROJO-VERDE-REFACTOR.md.
- **[Nuevo]** Ciclo 1: GET /api/tareas/resumen (total, pendientes, en curso, completadas y vencidas). Rojo: 4 tests fallando por TypeError y 404. Verde: 18 de 18. Refactor: helpers esActiva, estaVencida y contar.
- **[Nuevo]** Ciclo 2: GET /api/tareas/proximas?horas=N (activas que vencen pronto, por defecto 48 h, entre 1 y 720, límite inclusivo, ordenadas por fecha). Rojo: 17 tests fallando. Verde: 31 de 31.
- **[Mejora]** El resumen incorpora porVencer. El cambio de requisito se escribió primero como test (se actualizaron las expectativas del ciclo 1).
- **[Mejora]** Refactor del ciclo 2: venceEnVentana (sin condición duplicada), constantes en lugar de números mágicos, validarHoras en la capa de lógica y resultados del service renombrados a datos.
- **[Fix]** Detectado por el rojo: una ruta fija como /tareas/resumen se interpretaría como un id si se registra después de /tareas/:id. Las rutas fijas van primero.
- **[Nuevo]** Agentes de IA en .claude/agents/: revisor (el del TP 6, solo lectura) y tdd (guía el ciclo Rojo-Verde-Refactor y hace un commit por fase).
- **[Nuevo]** CLAUDE.md con comandos, arquitectura, flujo obligatorio y reglas del proyecto, y docs/FLUJO-DE-TRABAJO.md con el diagrama del flujo.
- **[Nuevo]** Revisión del diff con las cinco preguntas del revisor, aplicadas a mano (no se ejecutó el subagente): tres hallazgos de gravedad media, todos límites ya conocidos de la persistencia en archivo y de la falta de autenticación.
- **[Nuevo]** Documentación de la API: docs/API.md, backend/openapi.yaml (OpenAPI 3.0) y backend/requests.http para probar desde VS Code.
- **[Nuevo]** Tests del backend: de 14 a 31, todos correctos. Total del proyecto: 53 (31 backend y 22 frontend). Comprobación manual de los endpoints contra el servidor levantado.

Problemas conocidos a la versión actual

Tema	Detalle
Persistencia	El JSON completo se lee en cada request (también en resumen y proximas) y se reescribe en cada escritura; sin paginación ni bloqueo entre procesos. Aceptable para uso personal, no para muchos usuarios.
Filtros en la interfaz	La API ya filtra y ordena, pero la interfaz lo hace en el navegador sobre la lista completa.
Sin autenticación	No hay usuarios ni sesiones: cualquiera con acceso a la API ve todas las tareas.
Concurrencia	Dos escrituras simultaneas pueden pisarse (leer y reescribir todo el archivo).
Pruebas de interfaz	No hay tests de componentes ni de punta a punta; la UI no se probo manualmente en un navegador durante esta versión.
Zona horaria	Las fechas se guardan en UTC y se muestran en hora local del navegador; no se contemplan otras zonas.

Deudas anteriores resueltas

- Frontend incompleto: ahora hay App, lista, filtros y edición (v0.10.0).
- Mensajes de fecha unificados en el backend (v0.9.0).
- server.js abría el puerto al importarse: separado en app.js (v0.9.0).
- Comparación de duplicados asumía texto: ahora se válida el tipo (v0.9.0).
- Funcionalidades del TP 1 sin construir (editar, eliminar, buscar, filtrar, ordenar, recordatorios): implementadas (v0.9.0 y v0.10.0).

Próximos pasos sugeridos

- v0.12.0: que el frontend consuma /resumen y /proximas; tests de componentes (Testing Library) y prueba manual en navegador y celular.
- v0.13.0: base de datos real (SQLite) con índices, paginación y bloqueo de escrituras.
- v0.14.0: cuentas de usuario y autenticación.
- v1.0.0: despliegue, PWA para uso sin conexión y recordatorios configurables.